Pseudocode: 2D LiDAR SLAM (ICP, Mapping, Pose Graph)

(Derived from `slam.py`, `utilities/icp.py`, `utilities/features.py`, `utilities/mapping.py`, `uti`

Notation

Points are in $\mathbb{R}^2$ unless stated otherwise. Homogeneous poses are 3×3 matrices $T = \begin{bmatrix} R & t \\ 0 & 1 \end{bmatrix}$.
ICP returns a forward transform mapping *source* toward *target*: $p' = Rp + t$.

1 Test environment and supporting utilities

Experiments are run *offline* on recorded sensor logs rather than on a live network stream. The main entry point (`slam.py`) reads paths and timing options from `config.yaml`, then drives registration and mapping over a synthetic scan iterator built from disk. `services/lidar_service.py` implements that iterator: it parses semicolon-separated rows with a leading microsecond `timestamp` and flattened (x, y, z) triples per return, discards all-zero padding triples, and yields `(timestamp_raw, rel_time_` per line with optional sleep or file looping to mimic real-time acquisition; `rel_time_us` is microseconds since the first scan so lidar frames align with IMU time. When IMU-assisted initialisation is used, `services/imu_service.py` loads quaternion samples (each row: timestamp and q_x, q_y, q_z, q_w separated by semicolons), derives yaw about the vertical axis, and exposes `yaw_at(rel_time_us)` and `delta_yaw` by nearest-neighbour lookup, so the stack depends only on paired CSV logs and configuration, not on live middleware. Supporting scripts under `meta-utils/` are not part of the SLAM imports: they use PyVista for inspection—`pcplayer.py` plays files from `data/`, distinguishing simple comma-separated clouds from semicolon lidar scans, streaming frames in the background when helpful and applying stride or voxel downsampling for smooth playback; `pcview.py` visualises one or more static comma-separated clouds with optional legends and layer toggles; `pcman.py` applies rigid transforms to a cloud, overlays original and transformed points, and can export the result for checks on toy data such as `teapot.csv`.

2 Point cloud preprocessing

Downsampling

A single 2D or 3D LiDAR sweep often yields a very large number of points. Nearest-neighbour queries and repeated least-squares steps in ICP then dominate runtime and memory, and highly redundant samples along walls or ground add little new constraint while amplifying noise in dense regions. *Downsampling* reduces the point count before registration and mapping while preserving the geometric structure needed for alignment.

This pipeline uses *voxel grid downsampling*. Space is partitioned into axis-aligned cubes (squares in 2D) of fixed edge length v. All points falling in the same cell are merged into one representative. Here the representative is the *arithmetic mean* of the cell's points, which smooths jitter and keeps a stable centroid inside extended surfaces. The voxel edge length v controls the resolution–cost trade-off: smaller v retains more detail and tighter fits but increases the number of voxels and the cost of each ICP iteration; larger v speeds up matching and can improve robustness when scans are noisy or only loosely initialised, at the expense of losing thin

structures and fine alignment cues. Different stages may use different voxel sizes—for example a coarser grid for coarse rotation search or feature extraction, and a finer grid for scan-to-scan ICP and submap construction—so that expensive steps run on fewer points while the final alignment still uses a density matched to the sensor and the scene scale.

Voxel downsampling (mean per voxel)

Require: Point set $P = \{p_i\}_{i=1}^{N} \subset \mathbb{R}^d$, voxel edge length $v > 0$

$\ell \leftarrow \min_i p_i$ ▷ per-axis minima

for each point p_i **do**

  $k_i \leftarrow \lfloor (p_i - \ell)/v \rfloor$ ▷ integer voxel index

end for

Cluster points by identical k_i; for each cluster C, output $\frac{1}{|C|} \sum_{p \in C} p$

Ensure: Downsampled point set (one representative per occupied voxel)

Height filter and projection to 2D (`filter_and_flatten`)

Require: 3D points (x, y, z), bounds $z_{\min}, z_{\max}$

Keep rows with $z_{\min} \leq z \leq z_{\max}$; discard z

Ensure: 2D points (x, y)

3 Normals and ICP

2D normal estimation via local PCA (`estimate_normals_2d`)

Require: Points Q, neighbour count k

Build KD-tree on Q

for each point $q_i \in Q$ **do**

  $\mathcal{N}_i \leftarrow k{+}1$ nearest neighbours of q_i (including self)

  $C \leftarrow \mathrm{Cov}(\mathcal{N}_i)$ ▷ 2×2 covariance

  $(\lambda_{\min}, v_{\min}) \leftarrow$ smallest eigenpair of C

  $n_i \leftarrow v_{\min}/\|v_{\min}\|$ ▷ unit normal

end for

Ensure: Normals $\{n_i\}$

4 Feature-based pre-alignment

5 Occupancy mapping

6 Pose graph

Edge error (conceptually): predicted relative pose from $T(x_i)^{-1}T(x_j)$ compared to z_{ij} in SE(2).

7 SLAM orchestration (`run_slam`)

One point-to-line ICP step, 2D linearised (`_point_to_line_solve_2d`)

Require: Transformed source points $\{p_i\}$, target points Q, target normals $\{n_j\}$, match indices j_i

for each correspondence i **do**
  $q_i \leftarrow Q_{j_i}, \quad n_i \leftarrow n_{j_i}$
  Form row of A: $[c_i,\, n_{i,x},\, n_{i,y}]$ where $c_i = n_{i,y}p_{i,x} - n_{i,x}p_{i,y}$
  $b_i \leftarrow -\, n_i \cdot (p_i - q_i)$
end for
Solve least squares: $(A^\top A)x = A^\top b, \quad x = [\theta, t_x, t_y]^\top$
$R \leftarrow R(\theta)$ (true 2×2 rotation), $t \leftarrow [t_x, t_y]^\top$

Ensure: Incremental rotation R, translation t

Rigid transform from paired points — SVD / Procrustes

Require: Matched sets $\{p_i\}$, $\{q_i\}$ (same count), at least 2 pairs

$\bar{p} \leftarrow \text{mean}(p), \quad \bar{q} \leftarrow \text{mean}(q)$
$W \leftarrow \sum_i (p_i - \bar{p})(q_i - \bar{q})^\top$ ▷ or matrix form $(P - \bar{p})^\top (Q - \bar{q})$
$U, \Sigma, V^\top \leftarrow \text{SVD}(W)$
$R \leftarrow VU^\top$; if $\det(R) < 0$, flip sign of last row of $V^\top$ and recompute R
$t \leftarrow \bar{q} - R\bar{p}$

Ensure: Rotation R, translation t mapping $p \mapsto Rp + t \approx q$

Iterative Closest Point (`ICP`)

Require: Source S, target T, voxel size v, max iterations I, error threshold ϵ

Optional: initial guess (R_0, t_0); method (point-to-point or point-to-line); max correspondence distance $d_{\max}$
$S \leftarrow \text{VoxelDownsample}(S, v), \quad T \leftarrow \text{VoxelDownsample}(T, v)$
Initialise transformed cloud P from S and (R_0, t_0) if given; $R_{\text{tot}} \leftarrow I$, $t_{\text{tot}} \leftarrow 0$
if point-to-line in 2D **then**
  Precompute normals on T
end if
Build KD-tree on T
for $k = 1$ to I **do**
  for each point $p \in P$ **do**
    Find nearest $q \in T$ and distance d
  end for
  if $d_{\max}$ is set **then**
    Keep correspondences with $d^2 < d_{\max}^2$; abort if too few inliers
  end if
  if point-to-line (2D) **then**
    $(R, t) \leftarrow \text{PointToLineStep}(P, T, \text{normals}, \text{indices})$
  else
    $(R, t) \leftarrow \text{SVDProcrustes}(\text{centered inlier pairs})$
  end if
  $R_{\text{tot}} \leftarrow RR_{\text{tot}}, \quad t_{\text{tot}} \leftarrow Rt_{\text{tot}} + t, \quad P \leftarrow RP + t$ (row-wise)
  $e_k \leftarrow$ mean squared point-to-point distance to current nearest neighbours
  if $|e_k - e_{k-1}| < \epsilon$ **then**
    break
  end if
end for

Ensure: $R_{\text{tot}}, t_{\text{tot}}$, final error

Curvature proxy from local PCA (`compute_curvature`)

Require: Points P, neighbour count k

```
for each point p_i do
    N_i ← k+1 nearest neighbours
    λ_1 ≤ λ_2 ← eigenvalues of Cov(N_i)
    κ_i ← λ_1/(λ_2 + ε)
end for
```

Ensure: $\kappa_i \in [0, 1]$ (large $\approx$ corner-like)

Keypoint extraction with spatial non-max suppression (`extract_keypoints`)

Require: Points P, curvatures κ, count N_{kp}, minimum separation δ

```
Sort point indices by decreasing κ
K ← ∅
for each index i in sorted order do
    if |K| ≥ N_kp then break
    end if
    if every keypoint in K is at least δ away from P_i then
        Add i to K
    end if
end for
```

Ensure: Keypoint indices K

Sorted-distance descriptor (`compute_descriptors`)

Require: Full cloud P, keypoint indices K, descriptor size k

```
for each keypoint p ∈ P_K do
    d_1, ..., d_{k+1} ← distances to k+1 nearest points in P (including self)
    Descriptor ← sort([d_2, ..., d_{k+1}])        ▷ drop self-distance
end for
```

Ensure: Descriptor matrix

Descriptor matching with Lowe's ratio test (`match_descriptors`)

Require: Descriptors D^A, D^B, ratio $\rho \in (0, 1)$

```
for each row D_i^A do
    Find two nearest rows in D^B by squared Euclidean distance: j_1, j_2
    if ‖D_i^A − D_{j_1}^B‖² < ρ² ‖D_i^A − D_{j_2}^B‖² then
        Add match (i, j_1)
    end if
end for
```

Ensure: List of pairs (i, j)

RANSAC for 2D rigid alignment (`ransac_align`)

Require: Keypoints S, T, matches $\mathcal{M}$, iterations N, inlier threshold τ

```
for n = 1 to N do
    Randomly pick 2 distinct matches; estimate (R, t) by SVD Procrustes on the two pairs
    Count inliers: matches whose reprojection error ‖Rs + t − t_match‖ < τ
    Track best (R, t) with largest inlier count
end for
Refit (R, t) with SVD Procrustes on all inliers of the best model
```

Ensure: R, t, inlier count

Correlative rotation search (`rotation_search`)

Require: Scans S, T, voxel size, coarse step $\Delta_\theta^{\mathrm{c}}$, fine step $\Delta_\theta^{\mathrm{f}}$

Downsample S, T; centre S at origin, note centroids $\bar{s}, \bar{t}$
Build KD-tree on T
function SCORE(θ)
    $P \leftarrow R(\theta)S + \bar{t}$ ▷ rotate centred source, place at target centroid
    return mean squared distance from each point in P to its NN in T
end function
Coarse grid: $\theta \in [-180^\circ, 180^\circ)$ step $\Delta_\theta^{\mathrm{c}}$; pick θ^* minimising Score
Fine grid: θ in $[\theta^* - \Delta_\theta^{\mathrm{c}}, \theta^* + \Delta_\theta^{\mathrm{c}}]$ step $\Delta_\theta^{\mathrm{f}}$
$R \leftarrow R(\theta^*), \quad t \leftarrow \bar{t} - R\bar{s}$

Ensure: Initial R, t

Feature-based alignment pipeline (`feature_based_alignment`)

Require: Source S, target T, hyperparameters (voxel, k, N_{kp}, RANSAC, ...)

Downsample S, T
Keypoints from curvature + suppression on each cloud
Descriptors; matches via ratio test; RANSAC $\rightarrow (R, t)$ or failure

Ensure: R, t, inlier count

Scan pair: optional pre-alignment + ICP (`_run_icp_pair`)

if config uses rotation search and/or features **then**
    Optionally run `rotation_search`; optionally run `feature_based_alignment` on (pre-rotated) source
    Compose initial $R_{\mathrm{init}}, t_{\mathrm{init}}$
end if
Run `ICP`$(S, T, \ldots, R_{\mathrm{init}}, t_{\mathrm{init}})$

Bresenham line on the integer grid (`_bresenham`)

Require: Integer start (x_0, y_0), end (x_1, y_1)

Initialise Bresenham error and step directions (s_x, s_y)
while true **do**
    if current cell equals (x_1, y_1) **then break**
    end if
    Append current cell to list; advance cell per Bresenham midpoint rule
end while

Ensure: Cells on the segment from start toward end, *excluding* the endpoint

Log-odds occupancy update for one scan (`OccupancyGrid2D.update_scan`)

Require: Sensor origin o, hit points $\{h_i\}$ in world frame; priors $p_{\text{hit}}, p_{\text{miss}}$; clamp $[\ell_{\min}, \ell_{\max}]$

$\ell_{\text{hit}} \leftarrow \log \frac{p_{\text{hit}}}{1-p_{\text{hit}}}, \quad \ell_{\text{miss}} \leftarrow \log \frac{p_{\text{miss}}}{1-p_{\text{miss}}}$
Convert o and each h_i to grid indices
for each in-bounds hit cell **do**
    log_odds(cell) $+= \ell_{\text{hit}}$
end for
for each ray from grid origin to hit **do**
    for each cell c on Bresenham path *excluding* endpoint **do**
        log_odds(c) $+= \ell_{\text{miss}}$
    end for
end for
Clamp all log_odds to $[\ell_{\min}, \ell_{\max}]$

Replay map from pose history (`_rebuild_map`)

Require: Mapper M, list $(P^{(k)}, T^{(k)})$ of body points and global poses

Clear M
for each (P, T) **do**
    Origin $\leftarrow$ translation part of T; hits $\leftarrow T$ applied to P
    M.`update_scan`(origin, hits)
end for

Relative pose measurement between nodes (`relative_transform_vec`)

Require: Homogeneous poses T_i, T_j

return $\text{vec}(T_i^{-1} T_j)$ as $[\Delta x, \Delta y, \Delta\theta]^\top$

Gauss–Newton pose-graph optimisation (`PoseGraph2D.optimize`)

Require: Nodes $x_k = [x, y, \theta]_k$, edges (i, j) with measurement z_{ij} and information Ω_{ij}; fixed node index f

for iteration until max or convergence **do**
    $H \leftarrow 0$, $b \leftarrow 0$ (size $3n \times 3n$)
    for each edge (i, j) **do**
        $e, A, B \leftarrow$ error $e(x_i, x_j, z_{ij})$ and Jacobians $\partial e/\partial x_i$, $\partial e/\partial x_j$
        Accumulate $A^\top \Omega A$, $A^\top \Omega B$, $B^\top \Omega A$, $B^\top \Omega B$ into H; $A^\top \Omega e$, $B^\top \Omega e$ into b
    end for
    Anchor: replace block for node f in H with large diagonal, zero corresponding b
    Solve $H\,\delta = -b$; update each $x_k \mathrel{+}= \delta_k$ (wrap θ to $(-\pi, \pi]$)
end for

Accumulate ICP increment into global pose (`apply_incremental_pose_2d`)

Require: Current global pose T_{glob}, ICP output mapping previous scan $\rightarrow$ current: $p_{\text{cur}} \approx R p_{\text{prev}} + t$

$T_{\text{inc}}^{-1} \leftarrow \begin{bmatrix} R^\top & -R^\top t \\ 0 & 1 \end{bmatrix}$ ▷ inverse of forward body motion in 2D
$T_{\text{glob}} \leftarrow T_{\text{glob}}\, T_{\text{inc}}^{-1}$

Submap construction (`_build_submap`)

Require: Buffer of recent scans in global frame, voxel size v

Concatenate all points; **return** VoxelDownsample$(\cdot, v)$

Submap alignment: local rotation search + translation tweak (`_submap_rotation_search`)

Require: Current scan (local), submap (global), predicted pose T_{pred}, angle range/step, voxel size

Downsample; KD-tree on submap
Score rotations θ around predicted yaw with translation fixed to t_{pred} (mean squared NN distance)
Coarse then fine angular sweep; best R
One-step refinement: transform scan by R, match to submap by NN, keep bottom 80% distances by threshold, set t to mean of $(q - Rs)$ on inliers

Ensure: R, t for scan in world frame

Submap ICP (`_attempt_submap_icp`)

```
if IMU yaw available then
    Override predicted rotation with IMU; narrow angular search
else
    Wide angular search
end if
(R_0, t_0) ← submap rotation search result
Run point-to-point ICP with R_0, t_0 and max correspondence distance
```

Loop-closure candidate selection (`_find_loop_candidates`)

Require: Current pose index c, history of poses, distance threshold d, min scan gap g, max candidates, min cumulative travel L

```
Precompute cumulative path length along pose positions
for each past index i with c − i ≥ g do
    if ‖p_c − p_i‖ < d and path length from i to c is ≥ L then
        Add i as candidate
    end if
end for
Sort candidates by distance; return first few
```

Main online SLAM loop (high level)

```
Read lidar stream; optionally load IMU (yaw at time, delta yaw between scans)
for each scan do
    Optionally skip by process_every_n
    2D filter by height
    if first scan then
        Initialise map bounds, occupancy grid, pose graph root, submap buffer
    else
        Scan-to-scan: ICP with optional IMU rotation init or _run_icp_pair
        if error too large then skip pose update
        end if
        T_glob ← apply incremental transform
        if submaps enabled then
            Optionally correct T_glob if submap ICP agrees within thresholds
        end if
        Add pose-graph node; odometry edge from previous with information ∝ 1/error
        Insert scan into occupancy grid at T_glob
        if loop closure enabled then
            Find candidates; try ICP current vs old scan; if good, add loop edge z_lc, optimise
graph, rewrite history, rebuild submap buffer, _rebuild_map
        end if
    end if
end for
```

IMU yaw from quaternion (`_quat_to_yaw`)

Require: Unit quaternion (q_x, q_y, q_z, q_w)

yaw $\leftarrow \operatorname{atan2}\big(2(q_w q_z + q_x q_y),\ 1 - 2(q_y^2 + q_z^2)\big)$

IMU yaw lookup (`IMUService.yaw_at`)

Require: Sorted relative timestamps, yaw array, query time t

Binary search; compare neighbours; return yaw of nearest timestamp